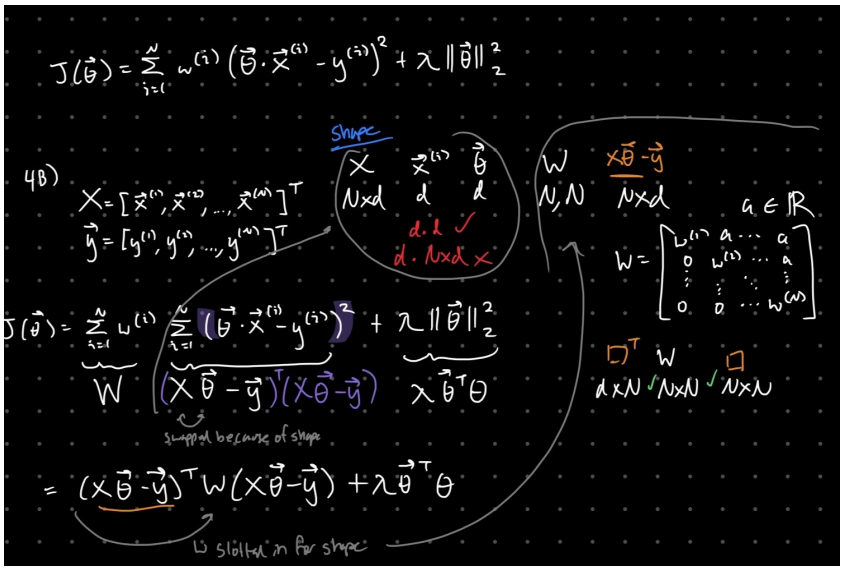
4 Weighted Linear Regression

4A)
Weighted examples would be useful in data that has lots of noise or outliers. By being able to weigh accurate data more 'heavily' it will help focus the algorithm to learn better.
4B)

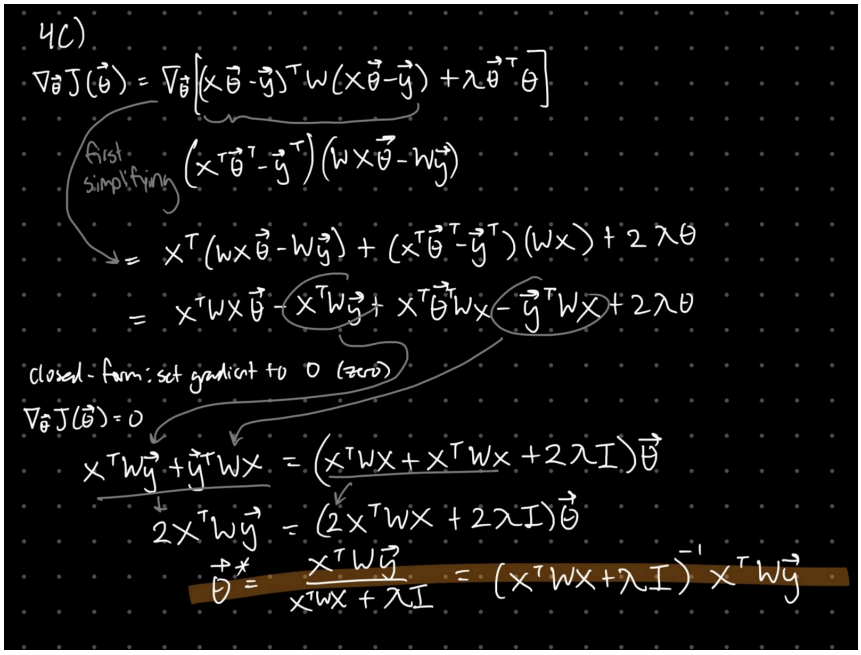
As established X represents the array containing x(i) vectors from x(i) to x(N), $\vec{y}$ contains the y(i) values and W is the diagonal weight matrix that contains the w(i) values.

Therefore the summation from 1 to N of $\vec{\theta} \cdot \vec{x}^{(i)} - y^{(i)}$ can be represented as $X\vec{\theta} - \vec{y}$. because it is squared there is 2 (one transposed to be able to multiply) and the w(i) is changed into a W resulting in the form: $(X\vec{\theta} - \vec{y})^T W(X\vec{\theta} - \vec{y})$. Likewise theta is also squared which results in the lambda times theta transposed and normal theta.

W is the diagonal weight matrix that contains the weight multiplier for each individual training example.



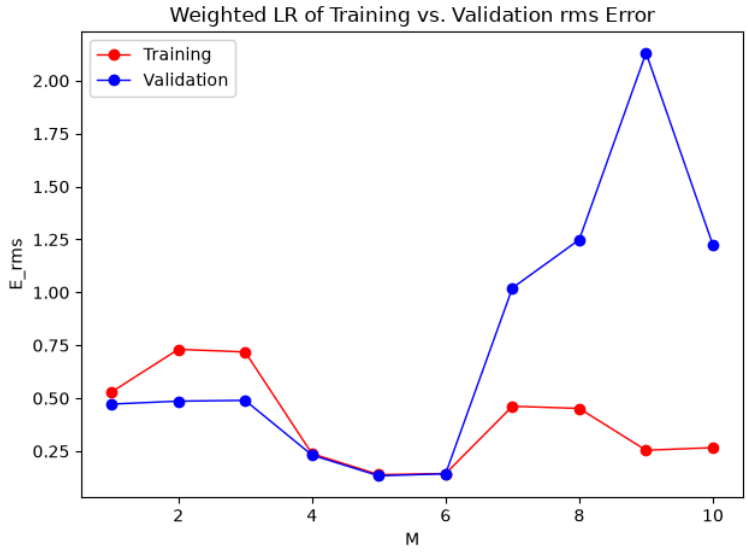
4C) $\vec{\theta} = (X^T W X + \lambda I)^{-1} X^T W \vec{y}$??



4D)??

```
# TODO: Implement this function
W = np.diag(weights)
tmp = np.linalg.inv(X.T @ W @ X + reg_param * np.eye(X.shape[1]))
theta = tmp @ X.T @ W @ y
return theta
```

4E)



Not the same conclusion, but generally the same. What is surprising is how even though the training data has plenty of errors, strangely the validated data actually performs better for degrees lower than 4. Then similarly degrees 4 to 6 are equally low (also strangely seeming

perfectly matching.) And the higher lambda's also overshoot, with the training having relatively low error despite not working for the validation data.
I think this occurs because of the integrated regularization within this function which helps balance the well fit and model complexity.